

InterLens Development Roadmap & Phase Plan

For the granular, step-by-step execution roadmap with CUDA/PyTorch optimization guidelines, see [PHASE_PLAN.md](#).

Phase 0: Setup & Packaging Infrastructure

- [] Initialize Python package layout (`pyproject.toml` , `interlens/`).
- [] Configure dependencies (`torch` , `transformer_lens` , `fastapi` , `uvicorn` , `pydantic`).
- [] Implement CLI entrypoint (`interlens launch`).
- [] Verify local PyTorch & TransformerLens model downloading and execution environment.

Phase 1: Model Adapter Abstraction

- [] Implement `BaseModelAdapter` abstract base class (`interlens/adapters/base.py`).
- [] Implement `GPT2Adapter` for GPT-2 family (`gpt2` , `gpt2-medium`).
- [] Implement LRU `SessionStore` (`interlens/server/session.py`) for caching activations.
- [] Add unit tests verifying adapter tokenization and activation cache shape standardizations.

Phase 2: First End-to-End Vertical Slice (Logit Lens)

- [] Implement `interlens/analysis/logit_lens.py` (unembedding layer activations).
- [] Implement FastAPI endpoints `/api/run` and `/api/analysis/logit-lens` .
- [] Build minimal embedded Web UI showing interactive Logit Lens table & ribbon chart.
- [] Verify end-to-end execution: Prompt input -> Model forward pass -> FastAPI -> Web UI rendering.

Phase 3: Core Feature Expansion

- [] **Residual Stream:** Implement L2 norm, variance, and PCA trajectory computation (`residual_stream.py`).
- [] **Attention Heads:** Implement attention matrix extraction, mini-grid payload, and arc diagram data builder (`attention_heads.py`).
- [] **Neuron Activations:** Implement top-K neuron ranking per token and token lighting strip generator (`neurons.py`).
- [] **Token Attribution:** Implement input token gradient & rollout attribution (`attribution.py`).

Phase 4: Model Family Expansion

- [] Implement `QwenAdapter` supporting Qwen2.5 (0.5B / 1.5B).
- [] Implement `LlamaAdapter` supporting Llama-3.2 (1B).
- [] Verify all Phase 3 analysis views work seamlessly across GPT-2, Qwen2.5, and Llama-3.2 without UI changes.

Phase 5: Advanced Causal Features

- [] **Activation Patching:** Implement clean vs. corrupt prompt activation swapping (`causal_patching.py`).
- [] **Causal Tracing:** Implement automated full-layer/position sweep & importance ranking (`causal_tracing.py`).
- [] **Model Comparison:** Implement dual-model execution sync and diff heatmap computation (`comparison.py`).

Phase 6: SAE Integration

- [] Integrate pretrained Sparse Autoencoder (SAE) downloading & feature lookup via Neuronpedia/EleutherAI indexes.
- [] Build SAE feature list panel & feature lighting strips (`sae_explorer.py`).

Phase 7: Polish, Documentation & Distribution

- [] Finalize UI polish, dark theme styling, micro-animations, and keyboard shortcuts.
- [] Add export functionality (PNG, SVG, JSON).
- [] Write comprehensive user documentation, tutorials, and Jupyter notebook integration examples.
- [] Package for distribution via PyPI (`pip install interplens`).